

WhatsApp AI Auto-Reply Agent

A platform-independent, Dockerised AI agent that automatically replies to WhatsApp messages using GPT-4o or a local Ollama LLM — powered by LangChain, FastAPI, and whatsapp-web.js.

Docker	Python 3.11	Node.js 20	LangChain	GPT-4o / Ollama
--------	-------------	------------	-----------	-----------------

1. Improved Project Prompt

The following is the refined, version-controlled prompt used to generate this entire project. It incorporates Docker for platform independence, explicit dependency versions, and clearer requirements around security, LLM switching, and deployment.

As an expert AI agent developer, design and implement a fully Dockerised, platform-independent WhatsApp auto-reply agent that runs on any OS (Linux, Windows, macOS) using a single **docker compose up** command. The system must meet the following requirements:

1. WhatsApp Integration

Use **whatsapp-web.js** (open-source, Node.js 20) to connect via QR-code session scanning. Filter incoming messages strictly to one configurable target contact (E.164 phone number). Ignore group messages. Persist the WhatsApp session across container restarts using a named Docker volume.

2. AI / LLM Backend

Use **LangChain** (Python 3.11) as the orchestration layer. Support two LLM providers switchable via a single field in **config.yaml** — no code changes required:

- **OpenAI GPT-4o** (cloud, API key via environment variable)
- **Ollama** (local, any model e.g. llama3, mistral — runs inside Docker)

Maintain a sliding conversation window (configurable turn count) for context memory.

3. Service Architecture

Two isolated Docker services managed by Docker Compose:

- **bridge**: Node.js 20 + whatsapp-web.js; forwards inbound messages via HTTP to the agent
- **agent**: Python 3.11 + FastAPI + LangChain; receives messages, calls the LLM, returns replies

All inter-service communication uses internal Docker networking (not exposed to host except agent:8000 on 127.0.0.1).

4. Configuration

A single **config.yaml** at the project root controls: target contact number, LLM provider and model, system prompt, memory window size, and service ports. The OpenAI API key must be supplied as an environment variable (never hard-coded), readable from a **.env** file.

5. Platform Independence

No host-level dependencies beyond Docker Desktop (or Docker Engine + Compose plugin). Chromium, Node.js, Python, and all libraries are installed inside containers via Dockerfiles. Setup must work identically on Fedora, Ubuntu, Windows 10+, and macOS.

6. Security

API keys sourced from environment only. Agent port bound to 127.0.0.1. WhatsApp session data excluded from version control via **.gitignore**. No message logging by default. Include an optional rate-limiter (max 1 reply per N seconds).

7. Deliverables

Provide: project folder structure, all source files (index.js, main.py, llm_chain.py), Dockerfiles for both services, docker-compose.yml, config.yaml template, .env.example, .gitignore, and a step-by-step setup guide covering first-run QR scan, LLM switching, and auto-start on laptop boot for Linux (systemd) and Windows (Task Scheduler).

2. System Architecture

The system is composed of two Docker containers orchestrated by Docker Compose, communicating over an internal network. No host-installed runtimes are required beyond Docker.

Service / File	Technology	Responsibility
bridge (Node.js)	whatsapp-web.js	QR session, message filter, HTTP forward
agent (Python)	FastAPI + LangChain	Receive message, call LLM, return reply
config.yaml	Project root (volume mount)	All runtime config — LLM, target, prompts
wwebjs_session	Named Docker volume	Persists WhatsApp auth across restarts
.env	Project root	OPENAI_API_KEY — never committed to git

Message flow

1. User sends WhatsApp message from the target phone number.
2. whatsapp-web.js (bridge container) receives it and checks against target_contact in config.yaml.
3. If matched, bridge POSTs the message body to http://agent:8000/message (internal Docker network).
4. FastAPI (agent container) passes the text to LangChain with conversation history.
5. LangChain calls GPT-4o (OpenAI API) or Ollama (local container) based on config.yaml.
6. LLM response returns through LangChain → FastAPI → bridge → msg.reply() back to WhatsApp.

3. Project Structure

```
whatsapp-agent/  
├── docker-compose.yml  
├── config.yaml  
├── .env.example  
├── .gitignore  
├── bridge/  
│   ├── Dockerfile  
│   ├── package.json  
│   └── index.js  
├── agent/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   ├── main.py  
│   └── llm_chain.py
```

4. Implementation Steps

Step 1 — Install Docker (the only host-level dependency)

This is the only thing you install on your laptop. Everything else (Node.js, Python, Chromium) lives inside containers.

OS	Command / Action
Fedora / RHEL	<code>sudo dnf install docker docker-compose-plugin sudo systemctl enable --now docker</code>
Ubuntu / Debian	<code>sudo apt install docker.io docker-compose-plugin sudo systemctl enable --now docker</code>
Windows 10/11	Download and install Docker Desktop from docker.com/products/docker-desktop
macOS	Download and install Docker Desktop from docker.com/products/docker-desktop

Step 2 — Create config.yaml

Place this file at the project root. It is mounted read-only into both containers.

```
# config.yaml
target_contact: "+966501234567" # E.164 format

llm:
  provider: "openai" # change to "ollama" for local
  openai_model: "gpt-4o"
  ollama_model: "llama3"
  ollama_base_url: "http://localhost:11434"

system_prompt: |
  You are a helpful assistant replying on behalf of the phone owner.
  Be concise and friendly.

memory_window: 10
bridge_port: 3000
agent_port: 8000
```

Step 3 — Create .env and .env.example

The .env file supplies secrets to Docker Compose. It must never be committed to version control.

```
# .env (real file – add to .gitignore)
OPENAI_API_KEY=sk-your-real-key-here
```

```
# .env.example (safe to commit)
OPENAI_API_KEY=sk-replace-me
```

```
# .gitignore
.env
.webjs_auth/
__pycache__/
node_modules/
```

Step 4 — docker-compose.yml

Defines both services, their build contexts, shared config volume, and the named session volume.

```
# docker-compose.yml
```

```
version: "3.9"

services:
  bridge:
    build: ./bridge
    volumes:
      - ./config.yaml:/app/./config.yaml:ro
      - wwabjs_session:/app/.wwabjs_auth
    depends_on:
      - agent
    environment:
      - AGENT_PORT=8000
    restart: unless-stopped

  agent:
    build: ./agent
    volumes:
      - ./config.yaml:/app/./config.yaml:ro
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
    ports:
      - "127.0.0.1:8000:8000"
    restart: unless-stopped

  volumes:
  wwabjs_session:
```

Step 5 — bridge/Dockerfile

Installs Node.js 20, Chromium (required by Puppeteer / whatsapp-web.js), and npm dependencies inside a Linux container — same on every host OS.

bridge/Dockerfile

```
FROM node:20-slim

RUN apt-get update && apt-get install -y \
  chromium --no-install-recommends \
  && rm -rf /var/lib/apt/lists/*

ENV PUPPETEER_SKIP_CHROMIUM_DOWNLOAD=true
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium

WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

CMD ["node", "index.js"]
```

bridge/package.json

```
{
  "name": "whatsapp-bridge",
  "version": "1.0.0",
  "dependencies": {
    "whatsapp-web.js": "^1.23.0",
    "qrcode-terminal": "^0.12.0",
    "axios": "^1.6.0",
    "js-yaml": "^4.1.0"
  }
}
```

Step 6 — bridge/index.js

The Node.js bridge: initialises the WhatsApp client, displays the QR code on first run, filters messages, and forwards them to the Python agent.

bridge/index.js

```
const { Client, LocalAuth } = require('whatsapp-web.js');
const qrcode = require('qrcode-terminal');
const axios = require('axios');
const yaml = require('js-yaml');
const fs = require('fs');

const cfg = yaml.load(fs.readFileSync('../config.yaml', 'utf8'));
const AGENT_URL = `http://agent:${cfg.agent_port}/message`;
const TARGET_JID = cfg.target_contact.replace('+', '') + '@c.us';

const client = new Client({
  authStrategy: new LocalAuth(),
  puppeteer: { args: ['--no-sandbox'], executablePath: process.env.PUPPETEER_EXECUTABLE_PATH }
});

client.on('qr', qr => {
```

```
console.log('Scan this QR in WhatsApp > Linked Devices:');
qrcode.generate(qr, { small: true });
});

client.on('ready', () => console.log('Bridge ready'));

client.on('message', async msg => {
  if (msg.from !== TARGET_JID || msg.isGroupMsg) return;
  console.log(`[IN] ${msg.body}`);
  try {
    const res = await axios.post(AGENT_URL, { from: msg.from, body: msg.body });
    const reply = res.data.reply;
    console.log(`[OUT] ${reply}`);
    await msg.reply(reply);
  } catch (err) { console.error('Agent error:', err.message); }
});

client.initialize();
```

Step 7 — agent/Dockerfile

agent/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

agent/requirements.txt

```
fastapi==0.111.0
uvicorn==0.29.0
langchain==0.2.0
langchain-openai==0.1.7
langchain-ollama==0.1.0
openai==1.30.0
pyyaml==6.0.1
python-dotenv==1.0.1
```

Step 8 — agent/llm_chain.py

Loads config, selects the LLM provider, builds the LangChain conversation chain with sliding window memory. Switching LLMs requires only editing config.yaml.

agent/llm_chain.py

```
import yaml
from langchain.memory import ConversationBufferWindowMemory
from langchain.chains import ConversationChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama

with open('../config.yaml') as f:
    cfg = yaml.safe_load(f)

def build_llm():
    if cfg['llm']['provider'] == 'openai':
        return ChatOpenAI(model=cfg['llm']['openai_model'], temperature=0.7)
    return ChatOllama(
        model=cfg['llm']['ollama_model'],
        base_url=cfg['llm']['ollama_base_url']
    )

PROMPT = PromptTemplate(
    input_variables=['history', 'input'],
    template=cfg['system_prompt'] + '\n\nHistory:\n{history}\n\nMessage: {input}\nReply:'
)

memory = ConversationBufferWindowMemory(
    k=cfg['memory_window'], human_prefix='User', ai_prefix='Assistant'
)

chain = ConversationChain(llm=build_llm(), prompt=PROMPT, memory=memory, verbose=False)
```

```
def get_reply(text: str) -> str:  
    return chain.predict(input=text)
```

Step 9 — agent/main.py

agent/main.py

```
from fastapi import FastAPI, HTTPException  
from pydantic import BaseModel  
from llm_chain import get_reply  
import yaml, uvicorn  
  
with open('../config.yaml') as f:  
    cfg = yaml.safe_load(f)  
  
app = FastAPI()  
  
class Msg(BaseModel):  
    body: str  
    model_config = {'populate_by_name': True}  
  
@app.post('/message')  
async def handle(msg: Msg):  
    if not msg.body.strip():  
        raise HTTPException(400, 'Empty message')  
    return {'reply': get_reply(msg.body)}  
  
if __name__ == '__main__':  
    uvicorn.run(app, host='0.0.0.0', port=cfg['agent_port'])
```

5. Running the Project

First-time setup

1. Clone / create the folder structure

```
mkdir whatsapp-agent && cd whatsapp-agent
```

2. Create config.yaml, .env, docker-compose.yml, and all source files as shown above

3. Build the Docker images (downloads all dependencies inside containers)

```
docker compose build
```

4. Start both services (add -d to run in background)

```
docker compose up
```

5. On first run the bridge prints a QR code — scan it in WhatsApp > Linked Devices

6. Send a test message from the target contact's phone

Switching to Ollama (local LLM)

No code changes are needed. Edit one line in config.yaml, then restart:

config.yaml — change provider

```
llm:
  provider: "ollama" # was "openai"
  ollama_model: "llama3"
```

Pull the model and restart

```
# On your host (Ollama must be running on the host or in its own container)
ollama pull llama3
docker compose restart agent
```

Useful Docker Compose commands

Command	Effect
docker compose up	Start all services (foreground)
docker compose up -d	Start all services (background / detached)
docker compose down	Stop and remove containers
docker compose logs -f bridge	Stream bridge (WhatsApp) logs
docker compose logs -f agent	Stream agent (Python) logs
docker compose restart agent	Restart agent after config change
docker compose build --no-cache	Rebuild images from scratch

6. Security Checklist

Concern	Mitigation
API key exposure	Sourced from .env only — never hard-coded; .env in .gitignore
Open network port	Agent binds to 127.0.0.1:8000 — unreachable from LAN
WhatsApp session	.webjs_auth/ stored in named Docker volume and in .gitignore
Message logging	Disabled by default; enable only if needed and store encrypted
Rate limiting	Add slowapi middleware to FastAPI for max 1 reply per N seconds
Container isolation	Each service runs in its own container with minimal Linux image

7. Auto-start on Laptop Boot

Linux — systemd user service

```
# ~/.config/systemd/user/whatsapp-agent.service
```

```
[Unit]
Description=WhatsApp AI Agent
After=docker.service

[Service]
WorkingDirectory=/home/youruser/whatsapp-agent
ExecStart=/usr/bin/docker compose up
ExecStop=/usr/bin/docker compose down
Restart=on-failure

[Install]
WantedBy=default.target
```

```
# Enable and start
```

```
systemctl --user enable --now whatsapp-agent
```

Windows — Task Scheduler

1. Open Task Scheduler and create a new Basic Task.
2. Set trigger to 'When I log on'.
3. Action: Start a Program.
4. Program: C:\Program Files\Docker\Docker\resources\bin\docker.exe
5. Arguments: compose -f C:\path\to\whatsapp-agent\docker-compose.yml up -d
6. Enable 'Run with highest privileges'.

8. Quick Reference

Stack summary

Component	Version	Role
whatsapp-web.js	1.23+	WhatsApp Web automation via Puppeteer
Node.js	20 (LTS)	Bridge service runtime
Python	3.11	Agent service runtime
FastAPI + uvicorn	0.111 / 0.29	HTTP server for the agent
LangChain	0.2	LLM orchestration and memory
langchain-openai	0.1.7	GPT-4o integration
langchain-ollama	0.1.0	Ollama / local LLM integration
Docker Compose	v2 plugin	Multi-service container orchestration

The session QR code only needs to be scanned once. After that, the named Docker volume persists the WhatsApp session so restarts are seamless. To revoke access, run docker compose down -v to delete the volume.